ArtConnect

Local Art Community Platform · Final Project Defense

MySQL · JavaFX · JDBC

3NF · 3-Tier Architecture

CRUD · Triggers · Views

Presented for Academic Oral Defense · 2026

Presentation Roadmap

What We Will Cover

01 **Project Overview**

Purpose, entities, and scope of ArtConnect

02 **Database Design — 40%**

CDM, LDM, 3NF normalization, integrity constraints

03 **Advanced SQL — 30%**

Triggers, stored procedures, views, transactions

04 **App Development — 20%**

3-tier architecture, JDBC integration, CRUD UI

05 **Design Justifications & Challenges — 10%**

Key decisions, thread synchronization fix, JDBC security resolution

Project Context

What is ArtConnect?

A **local art community platform** that bridges three distinct user groups through a relational database and a JavaFX desktop interface.

Artists

Profile · City · Disciplines · Artworks

Galleries

Physical Spaces · Exhibitions · Workshops

Community

Members · Bookings · Reviews

Core Entities: Artists, Artworks, Galleries, Exhibitions, Workshops, Bookings, Community Members, Reviews — implemented as a fully normalized relational schema in MySQL.

Methodology

Project Evolution — Steps 1 to 5

Steps 1 & 2 — Design & DDL

Conceptual Data Model (CDM) → Logical Data Model (LDM). Achieved 3rd Normal Form (3NF). Implemented DDL in MySQL with PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints.

Step 3 — Advanced SQL

Implemented: Trigger, Stored Procedure, View, ACID Transactions for secure workshop bookings, and query-optimization Indexes.

Step 4 — Java JDBC Integration

Migrated from In-Memory mock data to a live MySQL connection via JDBC. Established a 3-tier architecture: JavaFX UI → Singleton Service Layer → DAO Layer. Resolved the JDBC security constraint.

Step 5 — Full CRUD UI

Built JavaFX forms for complete Create, Read, Update, Delete operations on Artists. Resolved JavaFX UI thread synchronization using . All operations execute without runtime errors.

Database Design — CDM & LDM

📐 Conceptual Data Model (CDM)

- Entity identification: Artists, Artworks, Galleries, Exhibitions, Workshops, Members, Reviews, Bookings
- Relationship cardinalities: 1:N (Artist→Artwork), M:N (Exhibition↔Gallery)
- Semantic integrity rules defined before DDL

✅ 3NF Normalization Applied

- ✓
- ✓
- ✓

🗂️ Logical Data Model (LDM)

- CDM translated to relational tables with typed attributes
- M:N relationships resolved into junction tables (e.g., Bookings)
- All entities mapped to 3NF-compliant schemas

🔐 Integrity Constraints

- PRIMARY KEY on every table
- NOT NULL on critical fields
- CHECK (price > 0, status ENUM)
- FOREIGN KEY with ON DELETE
- UNIQUE (email, username)
- AUTO_INCREMENT surrogate PKs

Advanced SQL Features

⚡ Trigger — after_artist_insert

```
CREATE TRIGGER after_artist_insert
AFTER INSERT ON
FOR EACH ROW
BEGIN
  INSERT INTO

  VALUES  'INSERT'
END
```

Automatically logs every new artist insertion — enforces auditability without application-layer logic.

🔍 Stored Procedure — search_artworks

```
CREATE PROCEDURE search_artworks
            VARCHAR
                 DECIMAL


BEGIN
  SELECT   FROM
  WHERE
  AND
END
```

Encapsulates business logic server-side, reducing round-trips and centralising query maintenance.

👁 View — artist_portfolio

Joins Artists ⋈ Artworks to expose a read-only, pre-aggregated virtual table. Simplifies UI queries and enforces the principle of least privilege — no direct table access required.

🔒 Transactions & Indexes

Transactions: Workshop bookings wrapped in — guarantees ACID atomicity if capacity check fails.
Indexes: Created on , foreign keys to accelerate JOIN-heavy queries.

3-Tier Architecture

Tier 1 — Presentation

JavaFX UI
FXML Controllers
TableViews & Forms
User-facing layer

Tier 2 — Business Logic

Singleton Services
Validation Rules
Transaction Mgmt
Decoupled logic layer

Tier 3 — Data

JDBC DAOs
MySQL Database
PreparedStatements
Persistence layer

🔄 Migration: In-Memory → JDBC

Replaced all mock data structures with live DAO calls. The service layer's interface remained unchanged — the UI required zero refactoring, proving the power of separation of concerns.

✅ CRUD Operations on Artists

Full Create, Read, Update, Delete implemented via JavaFX forms. All 4 operations delegate to the DAO layer using — preventing SQL injection.

Application Development

JDBC Integration & DAO Pattern

Connection URL Configuration

```
// Fixed JDBC connection string
            "jdbc:mysql://localhost:3306/artconnect"
    "?useSSL=false"
    "&allowPublicKeyRetrieval=true"
    "&serverTimezone=UTC"
```

DAO — PreparedStatement Pattern

```
public void addArtist
"INSERT INTO Artists(name,city) VALUES(?,?)"

  try
     prepareStatement
       setString
       setString
       executeUpdate
```

🔑 Singleton Connection

A single class manages the JDBC connection pool, ensuring no duplicate connections are opened across service calls.

🧵 UI Thread Sync

Database calls run off the JavaFX Application Thread. UI updates are wrapped in to prevent errors and ensure thread safety.

🛡️ SQL Injection Prevention

All user inputs bound via parameters — eliminates injection attack vectors entirely.

Design Justifications & Challenges Solved

💡 Why 3NF?

3NF eliminates data redundancy and update anomalies. For a community platform with frequent artist profile edits and artwork status changes, this ensures a single source of truth. BCNF was considered but not required — our functional dependencies were straightforward.

💡 Why Stored Procedures over inline SQL?

Stored procedures centralise query logic at the database engine, reducing network overhead and preventing logic duplication across multiple application entry points. They also improve security by limiting direct table access.

🐛 Challenge 1 — JDBC Security Error

SOLVED ✓

Error:
Root Cause: MySQL 8 RSA key exchange disabled by default in newer JDBC connectors.
Fix: Added to the JDBC URL — a deliberate trade-off accepted for local development.

🐛 Challenge 2 — JavaFX Thread Synchronization

SOLVED ✓

Error:
Root Cause: Attempting UI updates from background JDBC threads.
Fix: All UI mutations wrapped in — textbook JavaFX concurrency pattern.

Evaluation Summary

Grading Criteria Coverage

Database Design — CDM, LDM, 3NF, Constraints — **40%**

Covered: CDM entities & cardinalities, LDM translation, 3NF proof, PK/FK/CHECK/UNIQUE constraints

Advanced SQL — Trigger, SP, View, Indexes, Transactions — **30%**

Covered: after_artist_insert trigger, search_artworks SP, artist_portfolio view, ACID transactions, FK indexes

Application Dev — 3-Tier, JDBC, CRUD, In-Memory → DB — **20%**

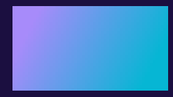
Covered: Entities, DAOs, Singleton Services, FXML controllers, full CRUD on Artists, zero runtime errors

Report & Documentation — Justifications & Challenges — **10%**

Covered: 3NF justification, SP rationale, JDBC security fix, JavaFX thread sync resolution

Total Coverage

All rubric criteria explicitly addressed across Slides 5–9



Thank You

ArtConnect demonstrates a complete software engineering lifecycle — from relational theory to a production-grade desktop application — built on principled design decisions at every step.

3NF Schema

Advanced SQL

3-Tier Java App

Bugs Resolved

Questions Welcome